

Build an Agent from Scratch

Assignment 3

Submitted by:

Amitai Kellerman - 313466112

Guy Kalati – 318366150

Part 1: Read a real agent

1- Agent & Version

We chose the NanoClaw agent in the latest release, the v2.1.17 version.

2- Architecture

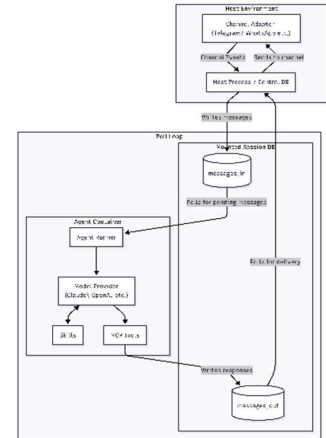
Details for this diagram are found in 'docs/architecture.md' under the "Core Idea" and "Message Flow" sections and 'docs/architecture-diagram.md'.

3- The Loop

The main reasoning loop is in 'container/agent-runner/src/poll-loop.ts' within the runPollLoop function.

The overarching poll loop runs indefinitely (while (true)) until the container is explicitly killed by the host (due to an idle timeout) or aborted. However, a single reasoning turn (processing a specific batch of messages) terminates when the provider's event stream yields a result event.

The runner itself implements a single-loop agent pattern. It operates as a continuous poll-query-respond loop isolated in its own container.



```
while (true) {
  if (config.signal?.aborted) return;
  const messages = getPendingMessages(isFirstPoll).filter((m) =>
    m.kind !== 'system');
  // ...

  const query = config.provider.query({
    prompt,
    continuation,
    cwd: config.cwd,
    systemContext: config.systemContext,
  });
  // ...

  const result = await processQuery(
    query,
    routing,
    processingIds,
    config.providerName,
    // ...
  );
}
```

4- Tools & Dispatch

Tools are defined using the MCP via JSON schema objects within an McpToolDefinition interface, as defined in 'container/agent-runner/src/mcp-tools/core.ts'. For example, the sendMessage tool defines properties like 'to' and 'text'.

NanoClaw uses MCP as both a client and a server. The provider SDK acts as the MCP client, natively dispatching tool calls to an internal MCP server spawned by the agent runner via Bun. Tool errors do not crash the agent;

they are explicitly surfaced back to the model (container/agent-runner/src/mcp-tools/core.ts).

```
export const sendMessage: McpToolDefinition = {
  tool: {
    name: 'send_message',
    description: 'Send a message to a named destination. If you have only one destination, you can omit `to`.',
    inputSchema: {
      type: 'object' as const,
      properties: {
        to: {
          type: 'string',
          description: 'Destination name (e.g., "family", "worker-1"). Optional if you have only one destination.',
        },
        text: { type: 'string', description: 'Message content' },
      },
      required: ['text'],
    },
  },
}
```

5- Memory

Conversation history is maintained via a push-based `MessageStream` class and automatically compacted by the SDK when exceeding the tokens threshold (`container/agent-runner/src/providers/claude.ts`). Episodic memory persisted by archiving `.jsonl` transcripts into markdown files via the `archiveTranscriptFile` function (`container/agent-runner/src/providers/claude.ts`); semantic and procedural memory are stored externally in the `/workspace/agent/CLAUDE.md` and `skills` folders, loaded at boot (`container/agent-runner/src/index.ts`). Procedural learning and skill creation occur through file-modification tools writing directly to the mounted agent folder, allowing the agent to persistently modify its own source and instructions (`docs/architecture.md`, "Core Properties" and "Flexibility Model" sections).

```
/**
 * Push-based async iterable for streaming user
 * messages to the Claude SDK.
 */
class MessageStream {
  private queue: SDKUserMessage[] = [];
  private waiting: (() => void) | null = null;
  private done = false;
```

6- Prompts, Planning, Evaluation

The core system prompt for the agent is defined in 'container/CLAUDE.md'.

It primarily enforces concise communication and dictates how the agent must manually organize its workspace, persist long-term memory, and utilize conversation history. This complex state and memory management is arguably a structural component that should be handled programmatically rather than relying on LLM prompt obedience.

The agent relies on implicit planning handled internally by the LLM via the providers SDKs which manages its own multi-turn reasoning and tool-use loops. Self-correction and reflection are delegated to the underlying SDKs.

The runs can be deeply traced and debugged using the built-in `/upload-trace` command, which extracts the session's raw `.jsonl` transcript and uploads it to a private dataset for analysis. Furthermore, the system logs routing decisions, errors, and progress events via a structured logger to both global host files (`nanoclave.log`, `nanoclave.error.log`) and isolated per-group container logs (`container/agent-runner/src/upload-trace.ts`, `src/log.ts`, `docs/agent-runner-details.md`).

```
You are a NanoClaw agent. Your name,
destinations, and message-sending rules are
provided in the runtime system prompt at the
top of each turn.
```

```
## Communication
\\ describes how to communicate with the user.

## Workspace
\\ describes workspace management.

## Memory
\\ describes the procedural memory management.

## Conversation history
\\ describes the episodic memory management.
```

7- Multi-Agent

NanoClaw coordinates multiple distinct agent groups, employing a manager-worker pattern where specialized agents handle specific tasks within shared or per-thread sessions. Agents communicate asynchronously via the standard database tables using the `send_to_agent` MCP tool, which routes a `messages_out` row that the receiving agent interprets as a regular chat message. There is a potential cost-explosion risk, but the architecture mitigates this through host-side permission validations and strict container concurrency limits (`docs/architecture.md`, `docs/agent-runner-details.md`, `docs/SPEC.md`).

8- What we would change

The architecture mounts the shared agent group directory (`/workspace/agent/`) with read-write permission across all concurrent sessions (`docs/architecture.md`), risking severe cross-session contamination if a compromised agent poisons shared skills or core instructions (e.g. `CLAUDE.md`). To establish provable security guarantees, the system should replace this vulnerability with multi-level access-control policies that mount shared workspaces as read-only for standard execution. Writing privileges to the shared environment must be strictly isolated and granted only to elevated administrative sessions.

Part 2: Building a Coding Agent (Harness) from scratch

The agent was able to complete all the tasks, as shown in the traces:

Task Number	Traces
1	<pre>[{ "role": "system", "content": "You are a coding agent. Use the provided tools to read, write, and run Python files in the sandbox to accomplish the user's goal. When a tool returns an 'error', read it and adapt. When the goal is met, reply with a short summary and no tool call." }, { "role": "user", "content": "List all files, read hello.py, run it, and tell me what it prints." }, { "role": "assistant", "content": "", "tool_calls": [{ "function": { "name": "list_files", "arguments": "{}" } }] }, { "role": "tool", "content": "[\"hello.py\", \"buggy.py\", \"notes.py\"]" }, { "role": "assistant", "content": "", "tool_calls": [{ "function": { "name": "read_file", "arguments": "{\"path\": \"hello.py\"}" } }] }, { "role": "tool", "content": "{\"path\": \"hello.py\", \"content\": \"print(\\\"\\\"hello world\\\"\\\")\\n\"}" }, { "role": "assistant", "content": "", "tool_calls": [{ "function": { "name": "run_python", "arguments": "{\"path\": \"hello.py\"}" } }] }, { "role": "tool", "content": "{\"path\": \"hello.py\", \"stdout\": \"hello world\\n\"}" }, { "role": "assistant", "content": "The file **hello.py** contains:\n\n```\npython\nprint(\"hello world\")\n\n\nWhen run, it prints:\n\n```\nhello world\n```" }]</pre>
2	<pre>[{ "role": "system", "content": "You are a coding agent. Use the provided tools to read, write, and run Python files in the sandbox to accomplish the user's goal. When a tool returns an 'error', read it and adapt. When the goal is met, reply with a short summary and no tool call." }, { "role": "user", "content": "Read buggy.py, identify the bug, fix it, run the fixed version, and confirm the output." }, { "role": "assistant", "content": "", "tool_calls": [{ "function": { "name": "read_file", "arguments": "{\"path\": \"buggy.py\"}" } }] }, { "role": "tool", "content": "{\"path\": \"buggy.py\", \"content\": \"def add(a, b):\\n return a - b # bug: should be a + b\\n\\nprint(add(2, 3))\\n\"}" }]</pre>

	<pre> }, { "role": "assistant", "content": "", "tool_calls": [{ "function": { "name": "write_file", "arguments": "{\"path\": \"buggy.py\", \"content\": \"def add(a, b):\\n return a + b\\n\\nprint(add(2, 3))\\n\"}" } }] }, { "role": "tool", "content": "{\"path\": \"buggy.py\", \"written\": true}" }, { "role": "assistant", "content": "", "tool_calls": [{ "function": { "name": "run_python", "arguments": "{\"path\": \"buggy.py\"}" } }] }, { "role": "tool", "content": "{\"path\": \"buggy.py\", \"stdout\": \"5\\n\\n\"}" }, { "role": "assistant", "content": "The file was read and found to contain a bug where the addition operation (`a + b`) was mistakenly replaced with subtraction. This has been fixed, and after re-running the corrected code, the output is **5**." }] } </pre>
3	<pre> [{ "role": "system", "content": "You are a coding agent. Use the provided tools to read, write, and run Python files in the sandbox to accomplish the user's goal. When a tool returns an 'error', read it and adapt. When the goal is met, reply with a short summary and no tool call." }, { "role": "user", "content": "Write a new file called reverse.py that defines reverse(s) and prints reverse('hello'), then run it." }, { "role": "assistant", "content": "", "tool_calls": [{ "function": { "name": "write_file", "arguments": "{\"path\": \"reverse.py\", \"content\": \"def reverse(s):\\n return s[::-1]\\n\\nprint(reverse('hello'))\\n\"}" } }] }, { "role": "tool", "content": "{\"path\": \"reverse.py\", \"written\": true}" }, { "role": "assistant", "content": "", "tool_calls": [{ "function": { "name": "run_python", "arguments": "{\"path\": \"reverse.py\"}" } }] }, { "role": "tool", "content": "{\"path\": \"reverse.py\", \"stdout\": \"olleh\\n\\n\"}" }, { "role": "assistant", "content": "The file `reverse.py` has been created with the function definition and printed output, and it was successfully executed. The reverse of 'hello' is **olleh**." }] </pre>
4	<pre> [{ "role": "system", "content": "You are a coding agent. Use the provided tools to read, write, and run Python files in the sandbox to accomplish the user's goal. When a tool returns an 'error', read it and adapt. When the goal is met, reply with a short summary and no tool call." }, { "role": "user", "content": "Search all files for the word 'TODO'. For every match, read that file and tell me its full contents." }, { "role": "assistant", </pre>

```

    "content": "",
    "tool_calls": [
      {
        "function": {
          "name": "search_files",
          "arguments": "{\"pattern\": \"TODO\\\"}"
        }
      }
    ]
  },
  {
    "role": "tool",
    "content": "{\"matches\": [{\"file\": \"notes.py\", \"line\": 1, \"text\": \"# TODO: add input validation\\\"}, {\"file\": \"notes.py\", \"line\": 2, \"text\": \"# TODO: write unit tests\\\"}]}"
  },
  {
    "role": "assistant",
    "content": "",
    "tool_calls": [
      {
        "function": {
          "name": "read_file",
          "arguments": "{\"path\": \"notes.py\\\"}"
        }
      }
    ]
  },
  {
    "role": "tool",
    "content": "{\"path\": \"notes.py\", \"content\": \"# TODO: add input validation\\n# TODO: write unit tests\\n\\n greet(name):\\n return f'Hello, {name}!'\\n\\\"}"
  },
  {
    "role": "assistant",
    "content": "The file **notes.py** contains the following content:\\n\\n``python\\n TODO: add input validation\\n TODO: write unit tests\\n\\n greet(name):\\n return f'Hello, {name}!'\\n```"
  }
]

```

Part 3: “Agent, earn me money pls! \$\$\$\$”

3A: Explore the exchange with the MCP Inspector

Tool Name	Arguments	Result
get_symbols	-	<pre>{ "items": [{ "name": "Apple Inc.", "symbol": "AAPL", "last_cents": 31339, "tick_cents": 1, "prev_close_cents": 31066 }, { "name": "Amazon.com Inc.", "symbol": "AMZN", "last_cents": 24362, "tick_cents": 1, "prev_close_cents": 24598 }, { "name": "Alphabet Inc.", "symbol": "GOOGL", "last_cents": 36192, "tick_cents": 1, "prev_close_cents": 36703 }, { "name": "IBM Corp.", "symbol": "IBM", "last_cents": 30205, "tick_cents": 1, "prev_close_cents": 30613 }, { "name": "Meta Platforms Inc.", "symbol": "META", "last_cents": 60312, "tick_cents": 1, "prev_close_cents": 61558 }, { "name": "Microsoft Corp.", "symbol": "MSFT", "last_cents": 38334, "tick_cents": 1, "prev_close_cents": 38884 }, { "name": "NVIDIA Corp.", "symbol": "NVDA", "last_cents": 20412,</pre>

		<pre> "tick_cents": 1, "prev_close_cents": 19693 }, { "name": "Tuttle Capital SPAC ETF", "symbol": "SPCX", "last_cents": 14830, "tick_cents": 1, "prev_close_cents": 14947 }] } </pre>
get_quote	symbol= AAPL	<pre> { "content": [{ "type": "text", "text": "{\\"symbol\\": \\"AAPL\\", \\"last_cents\\": 29308, \\"prev_close_cents\\": 29430}" }], "isError": false } </pre>
get_news	limit=5	<pre> { "items": [{ "at": "2026-07-01T17:24:19.517456+00:00", "id": 179993, "url": "https://finnhub.io/api/news?id=29b6799feaf609f5f08307ce4b1b9a0bf0fe0f2c24a6c19a609429fd6e1f61f8", "symbol": "SPCX", "headline": "6 Market Experts Sound Off On AI, Tech, Hot IPOs, And Top Stocks For 2026" }, { "at": "2026-07-01T17:14:18.731033+00:00", "id": 179921, "url": "https://finnhub.io/api/news?id=161470bf4167dae9e069f51a55a098de3224892232b90e86dd3940277418a0df", "symbol": "MSFT", "headline": "Explore the top gainers and losers within the dow jones index in today's session." }, { "at": "2026-07-01T17:13:18.697572+00:00", "id": 179909, "url": "https://finnhub.io/api/news?id=5c9a7cfe6c0fc975d626ccaae201391e32b905ec25080a48d19bedca5dd3bd6", "symbol": "META", "headline": "Wednesday's session: top gainers and losers in the S&P500 index" }, { "at": "2026-07-01T17:05:18.093226+00:00", "id": 179852, "url": "https://www.cnbc.com/2026/07/01/cramer-shares-next-steps-for-a-software-stock-after-a-rare-bullish-call-.html", "symbol": "MKT", "headline": "Cramer shares next steps for a software stock after a rare bullish analyst call" }, { "at": "2026-07-01T16:19:15.514235+00:00", "id": 179509, "url": "https://finnhub.io/api/news?id=697cf7e7b3892c56fa0459328f8fd2cb60cc6b111e59bf8c768aa3cbdf659a58", "symbol": "NVDA", "headline": "Where Will VOO Be in 20 Years? Here's What History Suggests." }] } </pre>
get_portfolio	Before buying	<pre> { "positions": [], "team_name": "BuffettBot", "cash_cents": 1000000, "open_orders": [] } </pre>
place_order	symbol= SPCX side =buy qty=1	<pre> { "fills": [{ "qty": 1, "price_cents": 16239 }], "status": "filled", "order_id": 101, "qty_open": 0, "qty_total": 1, "qty_filled": 1 } </pre>
get_portfolio	After buying	<pre> { "positions": [{ "qty": 1, "symbol": "SPCX" }], "team_name": "BuffettBot", "cash_cents": 9983753, "open_orders": [] } </pre>

3B: Wire your loop to the exchange

To help the agent, we added 3 local tools:

1. cents_to_dollars – for help converting the currencies and maintain the output format as defined in the system prompt.
2. actual_trade_price – to help the agent calculating the fees before trading.
3. calculator – to help the agent perform basic mathematical operations.

While the agent used the first two, it ignored the third tool entirely.

An evaluation of the agent's execution traces with granite4:micro model highlights a contrast in its operational capabilities.

Goal Adherence: The agent successfully reached the goals in most tasks. It failed only at goal 5 because of hallucinations, while hallucinating to buy quantity of 10000 stocks. In goals 1, 2 and 6 it managed to reach the goal despite using some redundant tools which may confuse the agent. See traces citations that support our claims below, and full traces in 'part3/granite traces' folder in the code.

Correctness: While the agent demonstrated good ability to reach the goals, the results were often incorrect. In goal 2 it identified IBM as the most expensive stock. In goals 3, 6 and 7 the cents to dollars conversion were incorrect and in goals 6 and 7 it hallucinated prices of stocks in addition. In goal 5 it didn't check the percentage at all and hallucinated quantity of 10,000 stocks, which was out of balance and therefore the trade failed. See traces citations that support our claims below, and full traces in 'part3/granite traces' folder in the code.

While the Granite model demonstrates a structural ability to navigate tool execution and complete most tasks, its operational reliability is severely undermined by redundant tool usage and execution hallucinations. Ultimately, the model's technical capacity to reach a goal is negated by its persistent factual incorrectness, characterized by catastrophic failures in basic arithmetic, currency conversions, and fabricated stock prices.

Goal Number	Supportive Traces
1 Portfolio report	<pre>[\\\ ... { "role": "tool", "content": "{\positions\": [{\"qty\": 2, \"symbol\": \"IBM\"}], \"team_name\": \"BuffettBot\", \"cash_cents\": 9939549, \"open_orders\": []}" }, \\\ ... { "role": "assistant", "content": "", "tool_calls": [{ "function": { "name": "get_symbols", "arguments": "{}" } }] }, \\\ ... { "role": "assistant", "content": "", "tool_calls": [{ "function": { "name": "get_quote", "arguments": "{ \"symbol\": \"AAPL\"}" } }] }, \\\ ... { "role": "assistant", "content": "***Portfolio Report**\n\n- **Cash (USD):** \$99,395.49 \n- **Positions:**\n - IBM \u2013 2 shares \n\nThe market price for IBM is at\u202f\u202f\u202f\$30,212\u202fcents per share (\u202f\u202f\u202f\$302.12 each)."</pre>

	<pre> }] </pre>
2 Most expensive and cheapest stock	<pre> [\\ ... { "role": "assistant", "content": "", "tool_calls": [{ "function": { "name": "get_symbols", "arguments": "{}" } }] }, { "role": "tool", "content": "{\n\"items\": [{\n\"name\": \"Apple Inc.\", \"symbol\": \"AAPL\", \"last_cents\": 31396, \"tick_cents\": 1, \n\"prev_close_cents\": 31066}, {\n\"name\": \"Amazon.com Inc.\", \"symbol\": \"AMZN\", \"last_cents\": 24208, \"tick_cents\": 1, \n\"prev_close_cents\": 24598}, {\n\"name\": \"Alphabet Inc.\", \"symbol\": \"GOOGL\", \"last_cents\": 35861, \"tick_cents\": 1, \n\"prev_close_cents\": 36703}, {\n\"name\": \"IBM Corp.\", \"symbol\": \"IBM\", \"last_cents\": 30213, \"tick_cents\": 1, \n\"prev_close_cents\": 30613}, {\n\"name\": \"Meta Platforms Inc.\", \"symbol\": \"META\", \"last_cents\": 60474, \"tick_cents\": 1, \n\"prev_close_cents\": 61558}, {\n\"name\": \"Microsoft Corp.\", \"symbol\": \"MSFT\", \"last_cents\": 38361, \"tick_cents\": 1, \n\"prev_close_cents\": 38884}, {\n\"name\": \"NVIDIA Corp.\", \"symbol\": \"NVDA\", \"last_cents\": 20227, \"tick_cents\": 1, \n\"prev_close_cents\": 19693}, {\n\"name\": \"Tuttle Capital SPAC ETF\", \"symbol\": \"SPCX\", \"last_cents\": 14572, \"tick_cents\": 1, \n\"prev_close_cents\": 14947}]]"} }, # according to this, META is the most expensive and SPCX is the cheapest. { "role": "assistant", "content": "", "tool_calls": [{ "function": { "name": "cents_to_dollars", "arguments": "{\n\"cents\": 35861}" # Hallucination. Not META nor SPCX value. } }] }, \\ ... # one more "cents to dollars" hallucinations. { "role": "assistant", "content": "", "tool_calls": [{ "function": { "name": "get_quote", "arguments": "{\n\"symbol\": \"AAPL\"}" # Hallucination + redundant. Not META nor SPCX and the cents appears in get symbols } }] }, \\ ... # one more "get quote" hallucinations. { "role": "assistant", "content": "", "tool_calls": [{ "function": { "name": "cents_to_dollars", "arguments": "{\n\"cents\": 24208}" } }] }, { "role": "tool", "content": "{\n\"\$242.08\""} }, { "role": "assistant", "content": "The current market snapshot shows the following prices:\n\n- **Most expensive:** IBM\u202fCorp., priced at **\$358.61** \n- **Cheapest:** SPAC ETF (SPCX), priced at **\$242.08**\n\nThese values are based on the latest \u201clast\u201d price for each symbol." }, # get the goal but hallucinate IBM as most expensive and the cents of SPCX] </pre>
3 Compare AAPL and MSFT	<pre> [\\ ... { "role": "assistant", "content": "The current quote for **AAPL** is a price of **\$31,415** (last cents\u202f=\u202f31,415). \n\nThe quote for **MSFT** is a price of **\$38,372** (last cents\u202f=\u202f38,372).\n\nUsing the 'pct_change' tool comparing MSFT\u2019s last\u2011price cents to AAPL\u2019s:\n\n- **23.52\u202f%** \u2013 MSFT\u2019s current share price is about **23.5\u202f%** higher** than AAPL\u2019s.\n\nSo MSFT is priced roughly 23.5\u202f% above its counterpart. No trade was placed." }, # All good but the cents to dollars convert (it did not use the dedicated tool).] </pre>
4 Summarize news	<pre> [\\ ... { "role": "assistant", "content": "Elon\u202fMusk loses \$50\u202fbillion in a single day; SpaceX moves bitcoin \u2013 **SPCX**. \n\nIs Tesla Stock a Buy as Deliveries Come in Ahead of Expectations and Its Robotaxis Launch in Miami? \u2013 **NVDA**. \n\nWhat Is SpaceX Really Worth? Wall Street Disagrees on Elon Musk\u2019s Huge Valuation \u2013 **MSFT**." }] </pre>

5 Buy stock	<pre>[\\\ ... <u># Good logic so far: read news -> get portfolio -> get quote for the stock it wants to buy (SPCX)</u> { "role": "assistant", "content": "", "tool_calls": [{ "function": { "name": "actual_trade_price", "arguments": "{\\"side\\": \"buy\\", \\"cents\\": 14648, \\"qty\\": 10000}" <u># Hallucinating qty</u> } }] }, \\\ ... { "role": "tool", "content": "{\\"error\\": {\\"\\\\"error\\\\"": \\"\\\\"insufficient cash: have 9939549, need 145752840 (incl fee 72840)\\\\"", \\"\\\\"status\\\\"": 409}\\"}" <u># Cause error</u> }, { "role": "assistant", "content": "I\u2019m unable to execute the buy because I don\u2019t have enough cash on hand (the trade would cost about **\$146,553.24**, roughly \$45,803\u202fm more than my current cash of \$9,939.54). \n\n**Summary:** \n- The most supportive recent news is about **SpaceX (SPCX)**. \n- My current net worth is **\$9,939.54**, and a 30\u202f% allocation would require buying roughly **5\u2011million shares** at ~\$14.50 each, totaling over \$146k\u2014well beyond my cash balance. \n- Therefore I must maintain at least 20\u202f% of the portfolio in cash (currently satisfied) but cannot buy this stock without additional capital.\n\nIf you\u2019d like to proceed later with more funds or a smaller allocation, just let me know!" } <u># Hallucinating numbers</u>]]</pre>
6 Sell stock	<pre>[\\\ ... <u># Good logic: get portfolio -> read news -> check the actual trade price of IBM stock</u> { "role": "assistant", "content": "", "tool_calls": [{ "function": { "name": "place_order", "arguments": "{\\"symbol\\": \"IBM\\", \\"side\\": \"sell\\", \\"qty\\": 2}" } }] }, \\\ ... { "role": "assistant", "content": "", "tool_calls": [{ "function": { "name": "get_portfolio", <u># Redundant. It already performed the sell.</u> "arguments": "{}" } }] }, \\\ ... { "role": "assistant", "content": "I sold the IBM position that was no longer justified, bringing my holdings down to cash\u2011only and resulting in a cash balance of **\$99,394.53** (the trade cost \$60.43 after fees). The recent news does not support holding IBM, so this sells complies with your goal." } <u># Good except the trade cost cents to dollars conversion.</u>]]</pre>
7 Check leaderboard	<pre>[\\\ ... { "role": "assistant", "content": "***Leaderboard Summary**\n\n **Your portfolio:** you own about **12,000 shares of AAPL**, worth roughly **\$23\u202fk** at the current price (**\$178.38 each**). Your cash balance is about **\$12\u202fk**. \n\n- **Net\u2011worth (cash + holdings):** approximately **\$35\u202fk**, giving you a rank near the middle of the list \u2013 currently between the \u201cstuff\u201d member\u201d and \u201cjoixsainteam\u201d teams.\n\n- **Top 5 net\u2011worth:** \n \u2013 1. pros\u2013 \$11,374,925 \n \u2013 2. aviv\u2013 \$10,913,458 \n \u2013 3. zenit\u2013 \$10,460,137 \n \u2013 4. Ron-Beiden\u2013 \$10,416,661 \n \u2013 5. elon-max\u2013 \$10,122,018 \n\nSo you sit comfortably above the bottom of the leaderboard but well below the very top performers. Keep an eye on market moves; a modest trade could boost your net worth further!" } <u># Hallucinating numbers, didn\u2019t check our portfolio. Did not convert cents to dollars correctly.</u>]]</pre>

LLaMA 3.1 8B Evaluation

We rerun the agent with llama3.1 8B model, which clearly showed where the model's limits are. Full traces are presented in the 'part3/llama traces' directory in the code.

Goal Adherence: In goals 1, 2, 3, 4 and 7 it reached the goal perfectly, without calling redundant tool calls or hallucinations. In the other goals it failed to reach the goal. In goals 5 and 6 the agent did not

reach the goal at all. It looks like it tries to perform some thinking before acting, and according to the hardcoded termination of the loop when no tool is called, it stops before any tool call.

Correctness: In goals 1, 2, 3 and 4 the response was correct, including cents to dollars conversions. In goal 3 it manages to do it even when it misused the `pct_change` tool, trying to pass string instead of integers. Although an error that specified the problem was given to the agent, it did not correct himself and tried to reuse the tool, but he performed the calculation correctly in his final response. In goal 7 it checked the leaderboard but didn't identify our team correctly and did not convert the cents to dollars as needed.

Model vs. Loop and Prompt: This test proves a fundamental rule about AI agents: the prompt gives the instructions, but the model's underlying training decides if it can do the job. Both models started from the same point – same prompts and tools, but llama outperform in terms of correctness. You can guide an agent's strategy with good prompt, but its actual success depends almost entirely on the LLM's ability to output correct, structured data, even if it makes a mistake along the way.

Granite vs. LLaMA: While the granite model reliably executes multi-step tool sequences at the cost of severe factual and arithmetic hallucinations, the llama model delivers superior reasoning and correctness but fails complex tasks due to premature loop termination.

Additional Goals

Because the agent failed in implicit buy goals, we tried one explicit, simpler, buy goal and another complex goal which checks the `get_trades` tool and involved buying:

1. Buy 2 stocks of SPCX and report the actual trade cost. Make sure you perform the trade.
2. What was the most expensive stock that I bought in the last 10 trades? Buy 1 share of the same symbol if it has supportive news.

We run the agent with both Granite and LLaMA models because we wanted to see how well they deal with explicit order to buy stock and if in the complex task they continue to act the same as in the other tasks:

1. Both models bought the correct stock and quantity but misreported the final cost. Granite utilized the `actual_trade_price` tool to correctly calculate total cents, whereas LLaMA bypassed it and ignored the multiple units. This demonstrate the importance of mathematical operation tools. Ultimately, both failed the cents-to-dollars conversion, outputting highly inaccurate figures (\$29,090 and \$14,534.00). In addition, as observed previously, Granite executed redundant steps before reaching the goal, while LLaMA completed the task directly in a single tool call.
2. In complex task, Granite achieved the goal, but it was incorrect, and LLaMA failed to reach the goal. Both failed to identify the right stock – IBM, and they chose SPCX as the most expensive stock. While Granite continues with the mistake until achieving the goal, LLaMA stopped early because of the loop termination condition.

Ultimately, these additional tests confirm that while the agent is reliable for single-step observations, it cannot be trusted with autonomous state modification or navigating conditional branches when it encounters friction.

Observed Limitation: Premature Loop Termination Due to Chain-of-Thought

A critical limitation observed during the runs, particularly with the llama 3.1 model, was the premature termination of the agent's execution loop. The model frequently engaged in Chain-of-Thought (CoT)

reasoning, outputting a strategic plan in natural language without attaching a formal JSON tool call. For example, in goal 5 the model output starts with: “To solve this goal, you need to first find which stock has the most supportive recent news. Then, see how much money I have vs what the last price is for that stock. ...”

The root cause is a structural mismatch between the model's conversational fine-tuning and the deterministic Python loop, which is hardcoded to terminate immediately upon receiving any response lacking a `tool_calls` payload. Consequently, the agent was cut off right after its planning phase, preventing execution.

To resolve this, the loop's stopping condition must be redesigned to accommodate intermediate reasoning. Instead of exiting immediately when a tool call is absent, the system should append the model's textual reasoning to the conversation history and trigger a subsequent generation step—perhaps by automatically injecting a hidden prompt such as, "Plan recorded. Now execute the necessary tool call." This would allow the agent to retain its generated context and successfully transition from planning to execution.

Conclusion

To conclude, we would not trust this agent with real money. Our evaluations show that the agent completely fails to execute complex goals regardless of the underlying model.

When using the Granite model, the agent handles the tool structure correctly but hallucinate numbers in both functions' parameters and in final response. Conversely, while the LLaMA model produces better arithmetic and less hallucinations, it fundamentally fails to achieve the goal due to early loop termination.

Since neither configuration can reliably complete a multi-step financial task without either inventing facts or stopping before reaching the goal, deploying this agent with real capital is completely unsafe.

AI Usage Disclaimer

To explore and analyze NanoClaw we used Gemini 3.1 Pro. It helped us to understand the agent's architecture and to find the relevant code lines.

As for parts 2 and 3, we used GitHub Copilot for understanding the codebase, code generation and debugging.

In addition, we used Gemini 3.1 Pro to formulate sentences and paragraphs in this report.